
Instructions for Submission

1. File Naming and Submission:
 - a. Create a **ZIP file** containing all your solution files.
 - b. Name the ZIP file as lab13.<ERP>.zip, where <ERP> is your ERP.
 - c. Ensure that the ZIP file contains:
 - i. C++ source files
 - ii. No unnecessary files (such as executables, temporary files, or editor configs)
 - d. Do not include TEXT(.txt) files.
2. Code Neatness:
 - a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
 - b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
 - a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
 - b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

A **linked list** is a linear data structure where elements (called nodes) are stored in **separate memory locations** and connected using pointers.

Unlike arrays:

- Elements do not require contiguous memory
- Accessing elements by index is not possible directly
- Insertions and deletions are efficient
- Traversal is required to access any element

Structure of a Node

```
struct Node {
    int data;        // value stored
    Node* next;     // pointer to next node
};
```

Visual representation of a node:

```
+-----+-----+
|  data  |  next  |
+-----+-----+
```

A chain of nodes:

[10] → [5] → [7] → [3] → nullptr

Creating a Linked List

```
Node* head = nullptr;    // start with empty list
Node* temp = new Node{5, nullptr};
head = temp;             // head now points to the new node
```

Visual:

```
head
↓
[5] → nullptr
```

New Keyword Usage in C++

In C++, the `new` keyword is used to dynamically allocate memory on the heap. Unlike stack allocation, memory allocated with `new` persists until it is explicitly deallocated with `delete`.

- `Node* n = new Node{value, nullptr};` Allocates a new node and returns a pointer to it.
- `delete n;` Frees the memory allocated previously to avoid memory leaks.
- Dynamic allocation allows flexible memory usage, which is essential for structures like linked lists where the number of elements is not fixed.

Core Linked List Operations

Statement	Explanation
<code>Node* head = nullptr;</code>	Creates an empty list.
<code>new Node{x, p};</code>	Creates a new node with value x, next pointer p.
<code>insertAtHead(x);</code>	Adds a new node before current head.
<code>insertAtTail(x);</code>	Traverses to end, then adds new node.
<code>deleteHead();</code>	Removes the first node.
<code>deleteValue(x);</code>	Finds node with value x and removes it.
<code>current = current->next;</code>	Moves to next node during iteration.

Insert at Head (Visual)

```
Node* temp = new Node{value, head}; head
= temp;
```

Before:

```
head
↓
[10] → [20] → nullptr
```

After inserting 5:

```
head
↓
[5] → [10] → [20] → nullptr
```

Insert at Tail (Visual)

```
Node* temp = new Node{value, nullptr};
Node* current = head;
while(current->next != nullptr) current
    = current->next;
current->next = temp;
```

Before:

```
head
↓
[10] → [7] → nullptr
```

After inserting 3:

```
head
↓
[10] → [7] → [3] → nullptr
```

Delete Head (Visual)

Before:

head
↓
[4] → [9] → [1] → nullptr

After deleteHead(): head

↓
[9] → [1] → nullptr

Delete a Specific Value

```
Node* current = head; Node*
previous = nullptr;

while(current != nullptr && current->data != value) {
    previous = current;
    current = current->next;
}

if(current != nullptr) {
    if(previous == nullptr) head = current->next;
    else previous->next = current->next;
    delete current;
}
```

Before deleting 7:

head
↓
[5] → [7] → [3] → nullptr

After deletion:

head
↓
[5] → [3] → nullptr

Iterating Through the List

```
Node* current = head;
while(current != nullptr) {
    std::cout << current->data << " ";
    current = current->next;
}
```

Visual:

current -> [10] -> [5] -> [3] -> nullptr

Lab Tasks

Task 1: Create a linked list

Creates a simple linked list with 3 nodes containing the values 1, 2, and 4. Use a **for** loop to traverse the list and prints each node's data.

Task 2: Print a list recursively

Write two recursive functions `print()` and `print()` that take the first node in a linked list and print the elements of a linked list in forward and reverse order, respectively.

You may not use any loops or iteration.

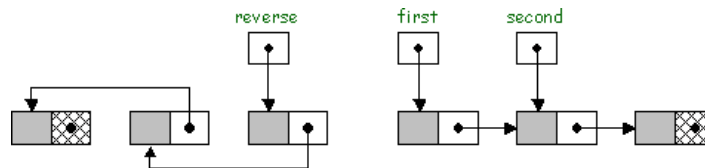
Task 3: Insert a node at the end

Write a function `insertEnd()` that takes the first node in a linked list and an integer value, and inserts a new node with the given value at the end of the list.

Task 4: Reverse Linked List (iteratively)

Write a non-recursive function that takes the first Node in a linked list as an argument, and reverses the list, returning the first Node in the result.

Solution: To accomplish this, we maintain references to three consecutive nodes in the linked list, `reverse`, `first`, and `second`. At each iteration we extract the node `first` from the original linked list and insert it at the beginning of the reversed list. We maintain the invariant that `first` is the first node of what's left of the original list, `second` is the second node of what's left of the original list, and `reverse` is the first node of the resulting reversed list.



You may not use recursion.

Task 5: Extract and Separate Nodes with Values Less than a Given Number

Write a function for a singly linked list named `findLessNodes(int num)`. This function should:

- take a pointer to the head of an existing linked list, and
- take an integer value `num`.

Your task is to traverse the original list and identify all nodes whose `data` value is **strictly less than** `num`. All such nodes must be:

1. removed from the original linked list, and
2. appended (in the same order) to a **newly created linked list**.

The function must return a pointer to the head of this new list. If no nodes contain a value less than `num`, return `nullptr`. The original list must remain unchanged except for the removed nodes.

Sample Output:

Original: [8] → [3] → [5] → [2] → `nullptr`

`num`: 5

New List: [3] → [2] → `nullptr`

Updated Original: [8] → [5] → `nullptr`

Original: [10] → [12] → `nullptr`

`num`: 5

New List: `nullptr`

Task 6: Remove Duplicates

You are required to design a singly linked list in which each node stores a **custom struct**. First, define a struct containing the following fields:

- `int erp;`
- `std::string studentName;`

This struct will serve as the `data` type of each node in the linked list.

Next, overload the `==` operator for this struct so that two struct objects can be compared directly. The comparison must consider the entire struct as a whole, not by checking `erp` and `studentName` separately.

After this, write a function `removeDuplicates` that:

- accepts a pointer to the head of a linked list containing the above struct,
- traverses the list and detects duplicate nodes by comparing the struct objects using the overloaded `==` operator,
- removes all duplicate nodes from the original list (keeping only the first occurrence), and
- returns the head pointer of the updated list.

You must ensure that the duplicate-removal logic is based solely on struct comparison via the `==` operator, not by manually comparing individual struct members.

Sample Output:

Original: {101,"Ali"} → {222,"Sara"} → {101,"Ali"} → `nullptr` Updated:

{101,"Ali"} → {222,"Sara"} → `nullptr`